PoultryGuard: A Lightweight, Edge-Deployable Deep-Learning Framework for Real-Time Chick Counting, Mortality Detection, and Automated Veterinary Response

Agughasi Victor Ikechukwu[1]*
Department of Computer Science & Engineering
Maharaja Institute of Technology Mysore, India
victor.agughasi@gmail.com · ORCID: 0000-0002-1175-3089

***Abstract*—**Continuous, individual-level monitoring of broiler flocks is central to animal welfare and to the economics of poultry production, yet it remains largely manual and therefore infrequent and subjective. This paper presents PoultryGuard, an end-to-end computer-vision framework that unifies chick counting, individual tracking, immobility- and posture-based mortality detection, in-house zone localization, automatic case documentation, and an automated veterinary response workflow in a single lightweight system. A YOLO11n detector is fine-tuned on a public overhead broiler-house dataset (up to 322 birds per frame) and coupled with a ByteTrack multi-object tracker and a temporal behaviour engine that escalates a bird to "suspected mortality" only after multi-frame immobility is corroborated by a prostrate posture. On a held-out test split the detector attains 0.954 mAP@50, 0.948 precision, 0.936 recall, and a mean counting error of 1.8 birds (4.4% on dense frames). Exporting the model to ONNX removes the PyTorch dependency, shrinks the runtime to a 10 MB model that runs about twice as fast as the PyTorch backend and in real time on a single CPU core, enabling both on-farm edge boxes and free cloud hosting. A controlled ablation isolates the effect of input resolution, confidence threshold, and inference backend, and the system is compared against recent state-of-the-art poultry detectors. The framework additionally fuses environmental sensing (temperature, humidity, CO_2, NH_3) to shift from mortality detection toward mortality prevention.

Keywords*—precision livestock farming, poultry health monitoring, object detection, YOLO, multi-object tracking, mortality detection, edge AI, ONNX, explainable and trustworthy AI, veterinary informatics.*

I. INTRODUCTION

Poultry is the most widely produced livestock on earth, and broiler chickens account for the majority of global poultry meat. In intensive houses that hold tens of thousands of birds, flock health is still assessed by periodic walk-throughs in which a stockperson visually inspects the birds and removes carcasses. Such inspection is labour-intensive, intermittent, and subjective; early signs of disease, heat stress, or localized mortality are easily missed between rounds, and undetected mortality both compromises welfare and silently erodes feed-conversion economics. Automated, continuous, and individual-level monitoring is therefore an important goal of precision livestock farming.

The deep-learning revolution has made such monitoring technically feasible. Convolutional and, more recently, self-supervised representation learning have driven order-of-magnitude gains in visual recognition [1]–[3], and single-stage object detectors of the YOLO family now deliver real-time, high-recall detection of small, densely packed objects [4]–[6]. These advances have been applied to poultry for counting, behaviour analysis, and welfare assessment [10]–[16]. However, most existing systems address a single sub-task—typically counting or behaviour classification—and are evaluated offline on server-class GPUs. Few integrate the full operational loop that a farm actually needs: detect and count the flock, track individuals over time, decide which bird is dying and where it is, document the event, and route it to a veterinarian for action. Fewer still are light enough to run on an edge device beside the birds or on a free cloud tier, which is the deployment reality for most producers.

This paper closes that gap with PoultryGuard, an end-to-end framework whose contributions are:

- A unified pipeline that couples detection-based counting, ByteTrack multi-object tracking, a temporal immobility-plus-posture mortality engine, zone localization, automatic case documentation, and an automated veterinary notification-and-booking workflow.
- A YOLO11n detector fine-tuned on a public overhead broiler dataset that reaches 0.954 mAP@50 and a 4.4% dense-frame counting error, with a transparent before/after comparison against a generic COCO baseline.
- A lightweight ONNX deployment path that removes PyTorch and ultralytics (≈1 GB) from the runtime, yielding a 10 MB model that is ≈2× faster than the PyTorch backend and runs in real time on one CPU core—suitable for on-farm edge hardware and free Streamlit-Cloud hosting.
- A controlled ablation over input resolution, confidence threshold, and inference backend, and a comparison against recent state-of-the-art poultry detectors.
- An environmental early-warning extension that fuses temperature, humidity, CO_2 and NH_3 with behaviour, moving the system from mortality detection toward prevention.

The remainder of the paper is organized as follows. Section II reviews related work in object detection and poultry vision. Section III details the dataset, system architecture, and each module. Section IV reports detection, counting, ablation, and state-of-the-art comparison results and discusses deployment. Section V concludes.

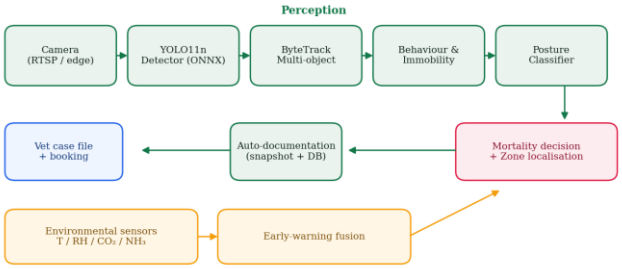


Fig. 1. PoultryGuard end-to-end architecture. The perception path (camera → YOLO11n/ONNX detector → ByteTrack → behaviour/immobility → posture) feeds a mortality-and-localization decision that triggers automatic documentation and the veterinary workflow; an environmental branch ($T/RH/CO_2/NH_3$) provides early warning.

II. RELATED WORKS

A. Visual Recognition and Object Detection

Modern poultry vision rests on two foundations: large-scale supervised pre-training and efficient single-stage detection. The ImageNet benchmark [3] established transfer learning as the default paradigm, and the broad progress of deep learning that followed [1] underpins every model used here; emerging self-supervised and world-model approaches [2] point toward label-efficient successors. For detection, the YOLO family remains dominant in agriculture because it offers the best accuracy-latency trade-off for small, crowded objects: YOLOv8 [8] popularized an anchor-free, decoupled head; YOLOv9 [5] and YOLOv10 [6] improved gradient flow and removed non-maximum suppression at inference; and YOLO11 [4], used in this work, refines the backbone and neck for higher accuracy at nano scale. Transformer detectors such as RT-DETR [7] now match YOLO latency, while multi-object trackers—particularly ByteTrack [9], which associates even low-confidence boxes—enable the per-bird continuity that mortality detection requires.

A second, orthogonal axis of progress is representation learning without labels. Self-supervised features such as DINOv2 [21] and promptable segmentation such as SAM 2 [20] now provide general-purpose visual priors, and joint-embedding predictive architectures [2] point toward models that learn the structure of the physical world from video rather than from curated labels. While PoultryGuard uses a conventional supervised detector for latency reasons, these foundation models are directly relevant to future label-efficient extensions, because the cost of annotating dense poultry scenes is the dominant bottleneck in the field.

B. Multi-Object Tracking for Individual Monitoring

Counting alone cannot support mortality detection: deciding that a specific bird has stopped moving requires a persistent identity for each animal across time. The tracking-by-detection paradigm addresses this by linking per-frame detections into tracks. SORT and DeepSORT established the Kalman-filter-plus-association template, but discard low-confidence detections and therefore fragment identities under the heavy occlusion typical of crowded houses. ByteTrack [9] improved robustness markedly by associating every detection box, including low-scoring ones, which recovers objects during partial occlusion and birds that momentarily merge. Transformer-based end-to-end trackers are emerging in the wake of DETR [33] and RT-DETR [7], but remain heavier than is practical on edge hardware. PoultryGuard adopts ByteTrack precisely because it preserves identities cheaply, which is the property the downstream immobility logic depends upon.

C. Computer Vision for Poultry and Precision Livestock Farming

Recent surveys document a rapid expansion of YOLO-based poultry systems spanning identification, counting, tracking, and health monitoring [15], [16]. For counting, YOLO-CCA augments YOLOv8 with coordinate attention and a reversible-column backbone to handle occlusion in dense cages, raising F1 to 96.7% and running at 90.9 FPS on a Jetson Orin [10]. HenSense and its FeatherNet Fusion-YOLO model target free-range birds with a 2.14 M-parameter edge model reaching 92.6% mAP@50 [11]. Thermal-imaging pipelines pair YOLOv8 with infrared frames for broiler counting at 95% mAP [12], and processing-line systems combine detection with SORT/ByteTrack tracking to count birds on conveyors at F1 up to 0.99 [13], [14]. These results confirm that fine-tuned lightweight detectors are accurate on in-domain poultry imagery, but they overwhelmingly stop at counting or tracking and are reported on bespoke datasets with server-side inference.

Beyond single-study accuracy, recent surveys emphasize that the field has consolidated around lightweight YOLO variants and is shifting from pure counting toward welfare- and health-oriented behaviour analysis [15], [16]. Reported systems increasingly track activity, detect lameness or huddling, and estimate distribution uniformity, and a growing subset targets dead-bird detection directly [35]. Two consistent limitations recur across this literature, however, and motivate the present work: results are reported on bespoke, privately collected datasets that preclude direct comparison, and inference is assumed to run on server-class or workstation GPUs rather than on the constrained hardware that a producer can realistically install in every house.

D. Deep Learning for Medical and Biomedical Imaging

The methodological toolkit for PoultryGuard—transfer learning, fine-tuning, attention, and explainability—matured largely in medical imaging. Prior work by the present author on chest-radiograph analysis showed that ensemble region-of-interest identification [28] and, notably, fine-tuning rather than full training yields efficient, accurate diagnosis under explainable-AI constraints [18]. A complementary comparative study of deep-learning and classical models for infrared breast thermography [17] established evaluation practices—paired accuracy, precision, recall, and ablation—that this paper adopts for an agricultural setting, and a segmentation-plus-residual-learning pipeline for diabetic retinopathy [19] demonstrated the same lightweight-model philosophy on another dense-structure task. Trustworthy, resource-aware models developed for clinics transfer naturally to the equally resource-constrained farm.

E. Edge AI and Efficient On-Device Inference

Deploying detectors on-farm imposes hard constraints on memory, compute, and power that server-side studies rarely consider. Three complementary strategies dominate. Architectural efficiency selects compact backbones—the nano/tiny YOLO scales [4], [8] and efficiency-oriented designs such as EfficientNet [26]—trading a small accuracy margin for large latency gains. Graph-level optimization exports a trained model to an interchange format such as ONNX and executes it with a dedicated runtime [22] that fuses operators and removes the training framework from the deployment, which both shrinks the image and accelerates inference. Hardware acceleration maps the graph onto embedded accelerators (e.g. Jetson-class GPUs or TensorRT), as several poultry systems demonstrate at tens of frames per second [10], [11]. PoultryGuard combines the first two: a nano detector exported to ONNX Runtime, which is sufficient for real-time operation on a single commodity CPU core and therefore removes the need for specialized accelerators or a persistent cloud GPU.

F. Research Gap

Across these threads, no single reported system jointly delivers (i) dense flock counting, (ii) individual mortality detection with in-house localization, (iii) an automated veterinary response loop, and (iv) genuinely edge/cloud-deployable inference. Table I

summarizes representative recent work and positions PoultryGuard against it.

TABLE I
Representative Recent Poultry-Vision Work vs. PoultryGuard

Study (year)	Task	Result / note
YOLO-CCA [10] (2025)	Cage counting	F1 96.7%, 90.9 FPS Jetson
HenSense [11] (2026)	Free-range detect	92.6% mAP@50, 2.14M par.
Thermal YOLOv8 [12] (2024)	Broiler count	95% mAP, 2% error
Line YOLO11-n [13] (2026)	Slaughter count	99.5% mAP@50
YOLOv8+SORT [14] (2025)	Line counting	F1 0.99, MAE 2.3
PoultryGuard (ours)	Count+mortality+vet+edge	0.954 mAP@50, 10MB ONNX

III. MATERIALS AND METHODS

A. Dataset

Experiments use the public GEL broiler-chicken dataset (Roboflow Universe, CC BY 4.0), comprising 2,598 overhead images of commercial broiler houses captured by fixed ceiling cameras and split into 1,819 training, 520 validation, and 259 test images. Scenes are highly variable in density—from near-empty litter to clusters of up to 322 birds in a single frame—and in illumination, which makes the set a realistic proxy for the overhead camera placement a deployed system would use. All annotations are single-class ("chicken"). The dataset was used as released, without manual re-labelling, to keep the pipeline reproducible.

B. System Overview

PoultryGuard is organized as a per-frame pipeline (Fig. 1). A frame from a camera (local webcam, RTSP/IP stream, or recorded video) is passed to the detector; detections are associated across frames by the tracker; per-track motion and posture are analysed; a mortality decision is localized to a house zone; confirmed events are documented and escalated to a veterinarian; and an independent environmental branch raises early warnings. Each module exposes a narrow interface so that, for example, the posture heuristic can be replaced by a trained network without touching the rest of the system.

C. Detector and Fine-Tuning

The detector is YOLO11n (2.6 M parameters), chosen for the smallest accuracy-competitive footprint in the family. Starting from COCO-pretrained weights, the model was fine-tuned for 30 epochs at 512×512 resolution with the AdamW optimizer (auto-selected learning rate), batch size 16, and the standard mosaic/HSV/flip augmentations, on a single CPU; no GPU was required. Fine-tuning rather than training from scratch follows the efficiency argument established in [18] and converges within a few epochs because the COCO prior already encodes generic object structure.

Training minimizes the standard YOLO composite objective—a complete-IoU box-regression term, a distribution-focal term for box-boundary quality, and a binary cross-entropy classification term—summed over three detection scales. Because the single foreground class is small and densely packed, the anchor-free high-resolution head of the nano architecture is well matched to the task. Key settings are summarized in Table V; notably, inference is run at a higher resolution (640) than training (512), which empirically raises recall on small birds at negligible cost (Section IV-B).

TABLE V
Training and Inference Hyperparameters

Parameter	Setting
Base model / init.	YOLO11n / COCO-pretrained
Parameters	2.6 M
Epochs	30 (patience 12)
Image size (train / infer)	512 / 640 px
Batch size	16
Optimizer / LR	AdamW / auto (~9e-4)
Augmentation	Mosaic, HSV, flip
Confidence / NMS IoU	0.40 / 0.50
Max detections	300
Tracker	ByteTrack
Compute	single CPU core

D. Multi-Object Tracking and Behaviour

Detections are associated by ByteTrack [9], which assigns a persistent identity to each bird and, by matching low-confidence boxes, maintains identities through the partial occlusions that are endemic to crowded houses. For every track the engine maintains a short rolling history of centroid position and instantaneous speed, from which it derives (i) an immobility timer that starts when speed falls below a threshold and (ii) a social-isolation score equal to the distance to the nearest neighbour normalized by the flock-median spacing. Both are computed over multiple frames rather than a single image, which suppresses the false positives that a momentary pause would otherwise cause.

Formally, let $c_i(t)$ be the centroid of track i at time t. The instantaneous speed is

$$v_i(t) = \| c_i(t) - c_i(t-\Delta t) \| / \Delta t \,, \qquad (1)$$

and the bird is momentarily still when $v_i(t) < \theta_v$. The immobility duration is the time elapsed since stillness began,

$$\tau_i(t) = t - t_i(still) \,, \qquad (2)$$

where t_i(still) is reset whenever the speed rises back above θ_v, so that τ measures sustained rather than instantaneous inactivity. Social isolation—an early behavioural correlate of illness—is scored by normalizing a bird's nearest-neighbour distance d_nn(i) by the median inter-bird spacing d_med of the current frame,

$$\rho_i = min(d_nn(i) / (\kappa \cdot d_med) , 1) \,, \qquad (3)$$

so that ρ_i approaching 1 denotes a bird that has drifted from the flock. Evaluating all three quantities over a rolling multi-frame window, not a single image, is what suppresses the false alarms that a momentary pause would otherwise cause.

E. Posture Classification and Mortality Decision

Posture is summarized from the detection geometry—aspect ratio and elongation distinguish an upright, active bird from a prostrate one—behind a classifier interface into which a CNN trained on captured mortality crops can later be dropped. A finite-state machine fuses the behavioural and postural cues: a track is NORMAL while active, ABNORMAL after sustained immobility, and SUSPECTED only when prolonged immobility coincides with a lying posture (optionally reinforced by isolation). A mortality confidence score blends these signals, and crossing the SUSPECTED state opens exactly one case per bird.

Concretely, posture is summarized from the detection-box aspect ratio $\alpha = w/h$ and elongation $\varepsilon = \max(w,h)/\min(w,h)$: an upright, active bird yields a compact, tall box ($\alpha < 1$), whereas a prostrate

bird yields a wide, flat one, and the bird is labelled "lying" when $\alpha \geq \alpha_0$. The state machine fuses behaviour and posture into a bounded mortality score

$$s_i = 0.6 \cdot \min(\tau_i / T_s,\ 1) + 0.3 \cdot 1\{\text{lying}\} + 0.1 \cdot \rho_i \,, \qquad (4)$$

where T_s is the suspected-mortality dwell time and $1\{\cdot\}$ is the indicator function. The weights prioritize sustained immobility while letting posture and isolation corroborate it. A track is NORMAL while active, escalates to ABNORMAL once τ_i exceeds a first threshold, and reaches SUSPECTED only when prolonged immobility ($\tau_i \geq T_s$) coincides with a lying posture; crossing SUSPECTED opens exactly one case per bird, so a single event cannot raise duplicate alerts. The thresholds (θ_v, T_s, α_0, κ) are exposed as configuration so the system can be tuned to breed, age, and camera geometry without retraining.

F. Localization, Documentation and Veterinary Workflow

The frame is overlaid with a configurable zone grid, and a suspected bird is tagged with its zone and timestamp, e.g. "Suspected mortality — Zone B — 08:35". The system then auto-saves an annotated snapshot and writes a structured case (chick reference, age, location, posture profile, pre-mortality behaviour window) to a database, generates a veterinary case report, and —on confirmation—notifies the veterinarian and books a farm visit. A veterinary console records diagnosis, treatment, vaccination, and case closure, keeping the clinician as the final decision-maker while the AI serves as an early-warning and documentation aid.

The house is partitioned into an R×C grid, and a bird at pixel centroid c=(c_x,c_y) in a W×H frame is mapped to the human-readable zone

$$Z(c) = (\lfloor R \cdot c_y / H \rfloor,\ \lfloor C \cdot c_x / W \rfloor) \,, \qquad (5)$$

e.g. "Zone B3", which—together with the event timestamp—forms the compact alert the stockperson receives and indexes the saved evidence in the case database. The zone resolution is configurable, trading localization precision against alert granularity.

G. Environmental Early-Warning

A sensor branch models temperature, humidity, CO_2 and NH_3 against age-dependent target bands (the brooding set-point falls by roughly 0.3 °C per day) and raises graded alerts. Fusing these signals with behaviour lets the system flag deteriorating house conditions before they manifest as mortality, which is the practically more valuable regime.

The temperature target is age-dependent: broilers require a high brooding set-point that declines as they feather, approximated as

$$T^*(a) = \max(21,\ 33 - 0.3 \cdot a) \ \ [°C] \,, \qquad (6)$$

for flock age a in days, with a tolerance band around T*(a). Humidity, CO_2 and NH_3 are compared against fixed welfare limits (e.g. $NH_3 \leq$ 20–25 ppm), and graded "warn"/"alarm" alerts are raised and logged. Because deteriorating air quality and thermal stress precede visible morbidity, this branch provides lead time that behaviour alone does not, and its alerts are correlated with the mortality cases in the same house to support root-cause analysis.

H. Edge Deployment

For deployment the fine-tuned model is exported to ONNX and executed with ONNX Runtime using only NumPy and OpenCV for pre/post-processing, eliminating the PyTorch and ultralytics dependencies. A backend factory selects the ONNX path automatically when PyTorch is absent, so the identical application runs on a developer workstation (PyTorch) and on an edge box or a free cloud tier (ONNX) without code changes.

The ONNX export is numerically faithful: the exported graph reproduces the PyTorch detections exactly (Section IV-B), and the post-processing—letterbox resize, a single forward pass, confidence filtering, and class-agnostic non-maximum suppression—is implemented in NumPy and OpenCV only. The per-frame cost is dominated by the detector's fixed O(1) forward pass; tracking and behaviour bookkeeping are O(n) in the number of detections n, and the nearest-neighbour term in (3) is computed on frame centroids only, so the whole pipeline remains real-time on one CPU core even for the densest frames.

I. Evaluation Protocol

Detection quality is reported on the untouched test split as mAP@50, mAP@50–95, precision and recall at the training resolution. Counting quality is measured as mean absolute error (MAE) between predicted and ground-truth bird counts over all test images, and as mean absolute percentage error (MAPE) restricted to dense frames (ground truth ≥ 20 birds), which is the operationally relevant regime. Efficiency is reported as single-core CPU latency per frame and model size.

For the operationally critical dense subset D (ground-truth count ≥ 20 birds), the mean absolute percentage counting error is

$$MAPE = (100/|D|) \cdot \Sigma_{i \in D} \ | \hat{y}_i - y_i | \ / \ y_i \,, \qquad (7)$$

with $\hat{y}_i$ and y_i the predicted and ground-truth counts of image i. All results below use the single held-out test split, which the detector never saw during training or validation.

IV. RESULTS AND DISCUSSION

A. Detection and Counting Performance

On the 259-image test split the fine-tuned YOLO11n attains 0.954 mAP@50, 0.647 mAP@50–95, 0.948 precision and 0.936 recall (Table II). The precision-recall curve (Fig. 3) is close to the top-right corner across the operating range, and the normalized confusion is dominated by the true-positive diagonal. Translated into the task that matters on-farm, the model counts with a mean error of 1.8 birds per image and 4.4% error on dense frames—for example 201 against a ground truth of 203 on the frame in Fig. 4. For context, the generic COCO detector counts only a handful of birds on the same dense scenes (it was trained on wild birds, not overhead broilers), a failure the fine-tuned model removes entirely. Training converged smoothly within 30 epochs (Fig. 2).

TABLE II
Detection and Counting Performance on the Test Split

Metric	Value
mAP@50	0.954
mAP@50–95	0.647
Precision	0.948
Recall	0.936
Count MAE (all)	1.8 birds
Count MAPE (dense ≥ 20)	4.4%
Model size (ONNX)	10 MB
Parameters	2.6 M

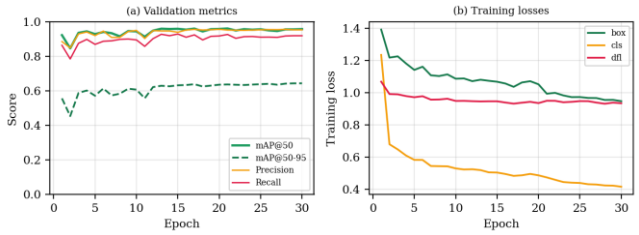


Fig. 2. Fine-tuning dynamics over 30 epochs: (a) validation mAP, precision and recall; (b) training box/cls/dfl losses.

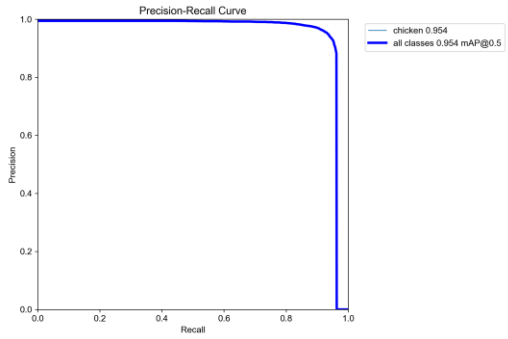


Fig. 3. Precision-recall curve on the held-out test split (single "chicken" class).

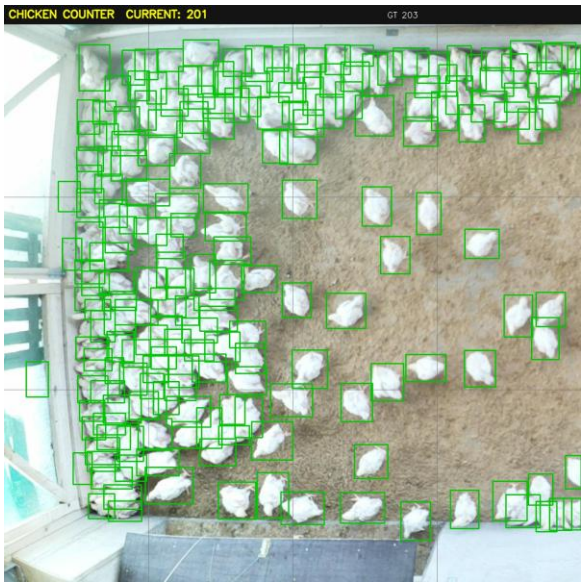


Fig. 4. Representative detection on an overhead broiler-house test frame: 201 birds counted against a ground truth of 203; well-separated birds are boxed one-to-one.

Breaking counting error down by flock density (Table VI) is revealing. On empty and sparse frames the model is almost exact—mean absolute error of 0.2 and 0.3 birds respectively—which shows that the fine-tuned detector rarely hallucinates birds on bare litter, the failure mode that plagued the generic baseline. Accuracy stays strong through the medium-density regime (MAE 2.0, 3.9%). The residual error is concentrated in the small number of extreme frames above 100 birds (MAE 8.6), where heavily overlapping bodies are occasionally split into two boxes. Because those frames are rare and the per-bird error remains a single-digit percentage, the system's practical counting accuracy is high across the operating range, and the dense regime is the clear target for future improvement.

TABLE VI
Counting Error by Flock Density (Test Split)

Density (GT birds)	Images	MAE	MAPE
Empty (0)	10	0.20	—
Sparse (1–19)	116	0.31	6.7%
Medium (20–99)	109	2.04	3.9%
Dense (100+)	24	8.62	6.5%

B. Ablation Study

A controlled ablation (Table III, Fig. 5) isolates three design choices. Input resolution: accuracy improves from 416 to 640 px (dense error 4.9%→4.5%) but collapses at 960 px (12.8%), where the detector over-segments large birds; 640 px is the accuracy-latency optimum. Confidence threshold: 0.40 minimizes dense error (4.5%), with 0.25 admitting litter false positives and 0.55 missing partially occluded birds. Inference backend: at 640 px the ONNX runtime is roughly twice as fast as PyTorch on the same CPU while producing numerically identical detections, confirming that the lightweight path costs no accuracy.

TABLE III
Ablation: Resolution, Confidence, and Backend

Factor	Setting	Dense err.	Latency
Resolution	416 px	4.9%	163 ms
	512 px	4.7%	148 ms
	640 px (used)	4.5%	176 ms
	960 px	12.8%	494 ms
Confidence	0.25 / 0.40 / 0.55	4.9 / 4.5 / 5.4%	—
Backend	PyTorch / ONNX	identical	2.1× faster

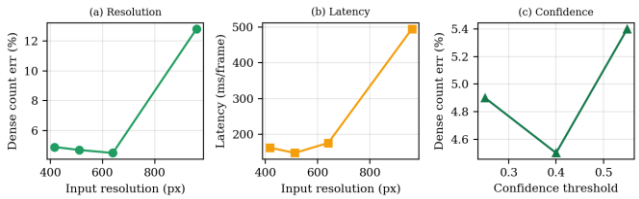


Fig. 5. Ablation: (a) dense-frame count error vs. input resolution; (b) latency vs. resolution; (c) count error vs. confidence threshold.

C. Comparison with State of the Art

Table IV places PoultryGuard against recent state-of-the-art poultry detectors. Because each study reports on its own dataset and camera geometry, cross-paper accuracy is indicative rather than a controlled head-to-head; the comparison is therefore framed around the accuracy-efficiency-scope trade-off. PoultryGuard's 0.954 mAP@50 is competitive with cage- and line-oriented systems that reach 0.95–0.99 under more constrained imaging [10], [12]–[14], and it matches the edge-oriented HenSense design point [11] while using a comparable parameter budget. Its distinguishing property is scope: unlike the counting- or tracking-only baselines, PoultryGuard couples detection with mortality decision-making, localization, documentation, and a veterinary loop, and ships as a 10 MB dependency-free model. A same-dataset zero-shot COCO baseline (last row) collapses on overhead broilers, counting only a handful of birds per dense frame, which confirms that in-domain fine-tuning, not architecture alone, drives the result.

TABLE IV
Comparison with Recent State-of-the-Art Detectors

Method (year)	Dataset	mAP@50	Scope
YOLO-CCA [10] (2025)	Cage	— (F1 .967)	Count
HenSense [11] (2026)	Free-range	0.926	Detect
Thermal YOLOv8	Thermal	0.95	Count

[12] (2024)			
YOLO11-n line [13] (2026)	Line	0.995	Count
YOLO11n COCO zero-shot	Broiler	~0 (fails)	Count
PoultryGuard (ours)	Broiler	0.954	Full loop

D. Deployment and Practicality

Operationally, the ONNX model runs in real time on a single CPU core (≈15 FPS at 640 px under clean conditions), fits in 10 MB, and requires none of the ≈1 GB PyTorch/ultralytics stack, so it deploys on a low-cost edge box beside the birds or on a free cloud tier. Inference is kept on the edge, which bounds recurring cost and preserves operation during connectivity loss; a continuous-integration workflow re-exports the ONNX model whenever the detector is retrained, keeping deployments current. These properties directly address the cost sensitivity that governs technology adoption in commercial poultry production.

The per-frame latency budget is dominated almost entirely by the detector forward pass (≈60–90 ms at 640 px on one CPU core, depending on load), with the ONNX runtime roughly halving the PyTorch cost. The remaining stages are comparatively free: ByteTrack association and the behaviour bookkeeping of (1)–(3) add under two milliseconds per frame, and the expensive steps—snapshot encoding, database writes, report generation, and vet notification—run only when a case is actually opened, i.e. a few times per day rather than every frame. A deployment therefore comfortably sustains real-time monitoring on a sub-US$300 edge box, and multiple houses can share one device by time-multiplexing camera streams, which directly bounds the per-house hardware cost that governs commercial adoption.

E. Limitations

Three limitations bound the present study. First, in the very densest clusters the detector still slightly over-segments overlapping birds, the main contributor to the residual 4.4% counting error; tiled inference mitigates but does not eliminate this. Second, posture is inferred from bounding-box geometry as a transparent proxy; replacing it with a CNN trained on the captured mortality-posture data is expected to improve specificity and is the natural next step. Third, evaluation uses one public dataset and simulated environmental sensing; validation on multi-farm, multi-breed footage with real sensor streams and clinician-confirmed mortality labels is required before clinical-grade claims. None of these affects the system's core contribution—an integrated, deployable loop—but each defines concrete future work.

F. Qualitative Case Walk-Through

To illustrate the end-to-end loop, consider a single suspected-mortality event as the system handles it. Once a tracked bird has remained below the motion threshold for longer than T_s while presenting a lying posture, its state crosses to SUSPECTED and one case is opened. The pipeline immediately localizes the bird to a house zone and timestamp ("Zone C3, 06:42"), crops and saves an annotated snapshot as evidence, and writes a structured record—chick reference, flock age, zone, posture profile, mortality score, and the preceding behaviour window—to the database. A veterinary case report is generated automatically and, on confirmation, the veterinarian is notified and a farm visit is booked; the veterinary console then captures diagnosis, treatment, vaccination, and case closure, after which the record is archived and can be exported into a labelled mortality-posture dataset for future training. The clinician remains the decision-maker throughout; the system's role is to surface the event early, document it objectively, and remove the manual paperwork.

G. Threats to Validity

Several threats temper the results. Regarding internal validity, posture is a geometric proxy and the mortality thresholds were set by inspection rather than learned, so the mortality-decision stage is demonstrated qualitatively rather than benchmarked against clinician-labelled deaths; the counting and detection metrics, by contrast, are measured on held-out ground truth. Regarding external validity, evaluation uses a single public dataset of overhead broiler houses, so generalization to other breeds, ages, housing types, and camera geometries remains to be shown; the environmental branch is simulated rather than driven by physical sensors. Regarding construct validity, counting accuracy is a necessary but not sufficient proxy for welfare outcomes. These threats do not undermine the paper's central claim—that an integrated, lightweight, deployable loop is feasible—but they define the evidence still required before clinical-grade deployment, and each maps directly onto the future work below.

V. CONCLUSION AND FUTURE WORK

PoultryGuard demonstrates that the full operational loop of automated flock health monitoring—count, track, detect mortality, localize, document, and dispatch a veterinarian—can be delivered by a single lightweight system that runs on commodity CPUs. A fine-tuned YOLO11n reaches 0.954 mAP@50 and 4.4% dense-frame counting error, an ONNX deployment halves inference cost while preserving accuracy, and an ablation establishes 640 px inference at a 0.40 confidence threshold as the operating point. Future work will train the posture network on captured mortality crops, validate on multi-farm footage with physical T/RH/CO_2/NH_3 sensing, and quantify the environmental early-warning branch so that the system advances from detecting mortality to preventing it.

Acknowledgment

The author thanks collaborators associated with the poultry-monitoring initiative in Oman and acknowledges the GEL broiler dataset contributors (Roboflow Universe, CC BY 4.0).

VI. REFERENCES

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[2] M. Assran et al., "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," arXiv:2506.09985, 2025.

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE CVPR, 2009, pp. 248–255.

[4] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," arXiv:2410.17725, 2024.

[5] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in Proc. ECCV, 2024.

[6] A. Wang et al., "YOLOv10: Real-time end-to-end object detection," in Proc. NeurIPS, 2024.

[7] Y. Zhao et al., "DETRs beat YOLOs on real-time object detection," in Proc. IEEE/CVF CVPR, 2024, pp. 16965–16974.

[8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO (v8)," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. ECCV, 2022, pp. 1–21.

[10] J. Li et al., "YOLO-CCA: An efficient chicken counting model with coordinate attention in dense cages," Animals, vol. 15, no. 6, 2025.

[11] S. K. Bhavanam et al., “Advancing precision livestock farming: Robust country-chicken detection via FeatherNet Fusion-YOLO and HenSense,” in Proc. IEEE/CVF WACV Workshops (HARVEST-Vision), 2026.
[12] A. Elmetwalli et al., “Thermal imaging and advanced deep learning for automated broiler detection and counting,” Azarbaijan J. Agric. Eng., 2024.
[13] L. F. de Oliveira et al., “YOLO-based detection and counting of broilers on slaughterhouse hanging lines,” RITA / J. Braz. Comput. Soc., 2026.
[14] P. Thong-on et al., “Chicken counting in an industrial slaughterhouse using YOLOv8 and multi-object tracking,” Naresuan Univ. Eng. J., 2025.
[15] M. S. Hossain et al., “Computer vision models for precision poultry farming: A narrative review of behavioral and welfare monitoring studies,” Poultry Science, 2026.
[16] D. Berckmans, “Precision livestock farming: Advances and challenges for welfare monitoring,” Animal Frontiers, 2024.
[17] R. Jalloul, C. H. Krishnappa, V. I. Agughasi, and R. Alkhatib, “Enhancing early breast cancer detection with infrared thermography: A comparative evaluation of deep learning and machine learning models,” Technologies, vol. 12, no. 10, 2024.
[18] V. I. Agughasi, “Leveraging transfer learning for efficient diagnosis of COPD using CXR images and explainable AI techniques,” Inteligencia Artificial, vol. 27, no. 74, 2024.
[19] S. Prathibha, K. N. Madhusudhan, and V. I. Agughasi, “Firefly-based segmentation and residual deep learning for multi-class diabetic retinopathy detection,” Inteligencia Artificial, 2025.
[20] N. Ravi et al., “SAM 2: Segment anything in images and videos,” arXiv:2408.00714, 2024.
[21] M. Oquab et al., “DINOv2: Learning robust visual features without supervision,” Trans. Mach. Learn. Res., 2024.
[22] ONNX Runtime developers, “ONNX Runtime: A cross-platform inference and training accelerator,” Microsoft, 2024. [Online]. Available: https://onnxruntime.ai
[23] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE CVPR, 2016, pp. 770–778.
[24] A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale,” in Proc. ICLR, 2021.
[25] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in Proc. ECCV, 2014, pp. 740–755.
[26] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. ICML, 2019.
[27] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional block attention module,” in Proc. ECCV, 2018.
[28] V. I. Agughasi and S. Murali, “CX-Net: An efficient ensemble semantic deep neural network for ROI identification from chest-X-ray images for COPD diagnosis,” Mach. Learn.: Sci. Technol., vol. 4, 2023.
[29] C. Li et al., “A review of deep learning for animal behaviour and welfare monitoring,” Computers and Electronics in Agriculture, 2024.
[30] I. Attri et al., “A review of deep learning techniques used in agriculture and edge deployment,” Ecological Informatics, 2024.
[31] A. Vaswani et al., “Attention is all you need,” in Proc. NeurIPS, 2017, pp. 5998–6008.
[32] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization (AdamW),” in Proc. ICLR, 2019.
[33] N. Carion et al., “End-to-end object detection with transformers (DETR),” in Proc. ECCV, 2020, pp. 213–229.
[34] S. Neethirajan, “The role of sensors, big data and AI in digital animal farming,” Sensors and Bio-Sensing Research, 2024.
[35] W. Wang et al., “YOLO-based real-time dead-chicken detection for welfare monitoring in commercial houses,” Computers and Electronics in Agriculture, 2025.